

ISTITUTO TECNICO SUPERIORE (ITS)

CORSO DI ALTA FORMAZIONE TECNOLOGICA IN SOFTWARE ENGINEERING E AI
DEVELOPMENT

TESINA DI FINE PERCORSO

Progettazione e Sviluppo di un Sistema RAG Enterprise Local-First con Garanzie di Sicurezza, Data Loss Prevention e Isolamento del Contesto: Il Progetto Hermes Knowledge

Candidato:	[Nome e Cognome]
Corso:	Tecnico Superiore per lo Sviluppo di Sistemi Software e Intelligenza Artificiale
Azienda Ospitante / Partner:	[Nome Azienda / Ragione Sociale]
Tutor Aziendale:	[Nome Tutor Aziendale]
Tutor Accademico/ITS:	[Nome Tutor ITS]

INDICE DEGLI ARGOMENTI

Capitolo 1 Introduzione e Contestualizzazione Industriale

- 1.1 La rivoluzione dell'Intelligenza Artificiale Generativa nel contesto aziendale
- 1.2 La problematica della frammentazione della conoscenza e l'allucinazione dei LLM
- 1.3 Obiettivi del progetto Ermes Knowledge

Capitolo 2 Analisi del Problema e Requisiti di Sistema

- 2.1 Limiti dei sistemi di ricerca tradizionali e dei chatbot generici
- 2.2 Requisiti funzionali e vincoli architetturali
- 2.3 Il principio dell'Evidenza e il paradigma Local-First

Capitolo 3 Architettura Software e Componenti Chiave

- 3.1 Panoramica dell'architettura del sistema Ermes
- 3.2 Backend in FastAPI e modularità dei servizi
- 3.3 Frontend in React e TypeScript per un'esperienza utente reattiva
- 3.4 Storage locale, vettori e gestione isolata delle biblioteche

Capitolo 4 Il Motore di Retrieval e Generazione (RAG)

- 4.1 Ingestion dei documenti, parsing e strategie di chunking dinamico
- 4.2 La Ricerca Ibrida: combinazione di vettori e keyword matching (BM25)
- 4.3 Il Re-ranker posizionale e l'algoritmo di deduplicazione Jaccard
- 4.4 Generazione controllata e vincolo all'evidenza documentale

Capitolo 5 Sicurezza, Data Loss Prevention (DLP) e Governance

- 5.1 Isolamento delle biblioteche e controllo degli accessi (RBAC)
- 5.2 Il filtro PII Guard: mascheramento in tempo reale di dati sensibili
- 5.3 Validazione algoritmica di Carte di Credito (Luhn), IBAN, Codici Fiscali e Token
- 5.3-bis Modello delle minacce e verifica per tentativo di violazione
- 5.4 Audit Logging e tracciamento delle operazioni conformi al GDPR

Capitolo 6 Qualità del Codice, Valutazione e Testing

- 6.1 L'importanza della verifica quantitativa: il Golden Set di valutazione
 - 6.1.1 Risultati della misurazione
 - 6.1.2 Il limite dell'astensione, misurato
- 6.2 Suite di test automatizzati (Pytest) e copertura delle regressioni
- 6.3 Rilevazione automatica delle lacune informative (Knowledge Gaps)

Capitolo 7 Esperienza di Tirocinio e Impatto Aziendale

- 7.1 Integrazione nel flusso di lavoro aziendale ed esperienza formativa
- 7.2 Impatto operativo dell'adozione di Ermes nell'organizzazione

Capitolo 8 Conclusioni e Sviluppi Futuri

- 8.1 Risultati raggiunti e considerazioni finali

8.2 Roadmap di evoluzione Enterprise (Scalabilità Cloud, Qdrant, OpenTelemetry)

CAPITOLO 1: INTRODUZIONE E CONTESTUALIZZAZIONE INDUSTRIALE

1.1 La rivoluzione dell'Intelligenza Artificiale Generativa nel contesto aziendale

Negli ultimi anni, l'avvento dei Large Language Models (LLM) ha trasformato radicalmente il modo in cui le organizzazioni interagiscono con le informazioni. L'abilità dei modelli linguistici di comprendere il linguaggio naturale, riassumere documenti complessi ed estrarre concetti chiave ha aperto scenari inediti per l'automazione dei processi e il supporto alle decisioni aziendali. Tuttavia, l'adozione di tali tecnologie all'interno di contesti strutturati ed enterprise presenta sfide primarie legate alla sicurezza, alla riservatezza delle informazioni e all'affidabilità delle risposte generate.

Le imprese moderne accumulano quotidianamente enormi moli di dati eterogenei: procedure operative, regolamenti interni, schede tecniche di prodotto, contratti, verbali di riunione e manuali di manutenzione. Questa conoscenza, anziché costituire un patrimonio accessibile e valorizzato, finisce spesso per rimanere frammentata all'interno di silos informativi, cartelle di rete condivise, archivi locali o piattaforme di messaggistica. Il tempo impiegato dai dipendenti per rintracciare l'informazione corretta ed aggiornata rappresenta un costo nascosto ma significativo per l'organizzazione.

1.2 La problematica della frammentazione della conoscenza e l'allucinazione dei LLM

L'impiego diretto dei modelli di intelligenza artificiale commerciali o generici per risolvere la frammentazione informativa presenta un ostacolo strutturale noto come "allucinazione". I modelli linguistici generativi sono progettati per calcolare la probabilità statistica delle parole successive all'interno di una sequenza, non per verificare la veridicità storica o fattuale delle affermazioni. Quando un LLM viene interrogato su aspetti specifici o dettagli di una procedura aziendale interna che non faceva parte del suo dataset di addestramento iniziale, tende a generare risposte apparentemente plausibili e con tono estremamente sicuro, ma totalmente prive di riscontro reale.

In un contesto industriale o direzionale, un'informazione errata o inventata può causare danni operativi, errori decisionali o violazioni di conformità normativa. Inoltre, l'invio indistinto di documenti aziendali riservati verso API di fornitori cloud terzi solleva gravi problematiche in materia di protezione dei dati personali, proprietà intellettuale e rispetto del Regolamento Generale sulla Protezione dei Dati (GDPR - UE 2016/679).

1.3 Obiettivi del progetto Ermes Knowledge

Il progetto Ermes Knowledge nasce con l'obiettivo di rispondere in modo rigoroso a queste criticità, realizzando una piattaforma avanzata di Retrieval-Augmented Generation (RAG) con approccio Local-First. Il principio guida del sistema è sintetizzato da una regola fondamentale: una risposta generata vale quanto l'evidenza documentale che la sostiene, e in assenza di un riscontro preciso all'interno dei documenti aziendali autorizzati, il sistema deve dichiarare espressamente l'assenza di evidenza anziché tentare di ipotizzare una risposta.

Concepito e sviluppato nell'ambito del percorso di Alta Formazione ITS in Software Engineering e AI Development, Ermes Knowledge integra funzionalità enterprise di livello avanzato, tra cui il

mascheramento preventivo delle informazioni sensibili (PII Guard), il controllo rigido degli accessi basato sui ruoli (RBAC), il tracciamento dei log di audit, l'isolamento logico tra differenti biblioteche documentali e un motore di ricerca ibrido con re-ranking posizionale e deduplicazione dei contenuti.

CAPITOLO 2: ANALISI DEL PROBLEMA E REQUISITI DI SISTEMA

2.1 Limiti dei sistemi di ricerca tradizionali e dei chatbot generici

I sistemi di ricerca documentale tradizionali presenti nella maggior parte degli ambienti aziendali si basano prevalentemente su tecniche di corrispondenza sintattica delle parole chiave (keyword search). Questo approccio, seppur veloce ed economico, mostra forti limiti quando l'utente formula una query utilizzando sinonimi, parafrasi o espressioni concettualmente equivalenti ma prive dei termini esatti contenuti nel testo originale. D'altro canto, i chatbot generici basati esclusivamente su LLM superano il limite semantico ma difettano di ancoraggio contestuale e trasparenza, non fornendo alcun riferimento verificabile alla fonte originale dell'informazione.

L'architettura Retrieval-Augmented Generation (RAG) nasce per coniugare il meglio dei due mondi: la capacità di ricerca concettuale e sintattica all'interno di una base di conoscenza privata e la capacità del modello linguistico di sintetizzare il contenuto recuperato in una risposta discorsiva e chiara. Tuttavia, affinché un sistema RAG sia idoneo all'impiego aziendale, occorre che la fase di recupero (retrieval) sia guidata da metriche di pertinenza rigorose e che l'accesso ai documenti rispetti rigorosamente le politiche di sicurezza e riservatezza dell'organizzazione.

2.2 Requisiti funzionali e vincoli architetturali

Durante la fase di analisi dei requisiti per il progetto Ermes Knowledge, sono state individuate le seguenti esigenze primarie:

In primo luogo, l'infrastruttura deve garantire che l'elaborazione dei documenti e la generazione dei dati avvengano primariamente all'interno del perimetro controllato dall'organizzazione, eliminando la dipendenza obbligatoria da servizi cloud esterni e prevenendo la fuga di dati riservati. In secondo luogo, il sistema deve consentire la creazione di biblioteche documentali distinte e separate, garantendo che gli utenti possano consultare esclusivamente i documenti afferenti ai gruppi o ai ruoli a cui appartengono.

Ogni affermazione prodotta dall'assistente virtuale deve essere accompagnata da citazioni puntuali e verificabili, consentendo all'utente di accedere al documento sorgente con un singolo click e verificare l'estratto di testo su cui si basa la risposta. Prima che qualsiasi testo venga elaborato o inviato ai modelli di linguaggio, il sistema deve analizzare il contenuto per identificare ed oscurare automaticamente informazioni sensibili come codici fiscali, numeri di carte di credito, coordinate IBAN, token di autenticazione e chiavi API.

Infine, la piattaforma deve mettere a disposizione degli amministratori strumenti di monitoraggio per tracciare le ricerche effettuate, analizzare le domande a cui il sistema non ha trovato risposta (Knowledge Gaps) e consultare i log di audit per scopi di conformità e sicurezza.

2.3 Il principio dell'Evidenza e il paradigma Local-First

La filosofia progettuale di Hermes Knowledge è fondata sul principio "Evidenza prima della generazione". Nel flusso di elaborazione di Hermes, il modello linguistico non viene utilizzato come fonte di conoscenza, bensì come un lettore e sintetizzatore di testo estremamente qualificato. Quando l'utente invia una domanda, il sistema esegue preventivamente una ricerca approfondita all'interno della biblioteca documentale selezionata. Solo i frammenti di testo che superano una soglia di pertinenza prefissata vengono forniti al modello all'interno del prompt di contesto.

Se la ricerca non produce alcun frammento di testo sufficientemente pertinente, il sistema interrompe la catena di generazione prima ancora di consultare l'LLM, restituendo all'utente un messaggio chiaro di assenza di evidenza. Questo approccio riduce i costi computazionali e impedisce che l'assistente risponda attingendo ai dati di addestramento del modello, che possono essere obsoleti o in contrasto con le direttive aziendali.

È importante precisare fin da subito il limite di questa garanzia, perché è stato misurato e non presunto: il principio elimina le allucinazioni *generative*, cioè le risposte inventate dal modello, ma non garantisce da solo che l'evidenza selezionata sia pertinente. Il capitolo 6 riporta la misura di questo scostamento e la contromisura adottata.

CAPITOLO 3: ARCHITETTURA SOFTWARE E COMPONENTI CHIAVE

3.1 Panoramica dell'architettura del sistema Hermes

L'architettura del progetto Hermes Knowledge è stata progettata seguendo i principi della modularità, del disaccoppiamento dei componenti e della manutenibilità nel tempo. Il sistema si compone di un layer di frontend reattivo sviluppato in Single Page Application, un backend in microservizi basato su FastAPI, un motore di gestione dello storage e dell'indicizzazione dei documenti, e un modulo di integrazione con i modelli linguistici (tramite runtime locali come Ollama o provider compatibili con lo standard OpenAI).

Tutti i componenti sono containerizzati tramite Docker e orchestrati mediante Docker Compose, garantendo la portabilità immediata del sistema su qualsiasi ambiente di sviluppo, staging o produzione senza dipendenze dall'ambiente host.

3.2 Backend in FastAPI e modularità dei servizi

Il backend è sviluppato in Python utilizzando il framework FastAPI, scelto per le sue elevate prestazioni assicurate dall'infrastruttura asincrona ASGI (Starlette e Pydantic), per la validazione automatica dei dati in ingresso e in uscita e per la generazione nativa della documentazione interattiva OpenAPI / Swagger.

La struttura del codice backend è organizzata in moduli funzionali altamente coerenti:

- Il modulo API gestisce gli endpoint REST, la gestione delle sessioni utente, l'autenticazione tramite chiavi API e token JWT, e l'esposizione delle risorse documentali.
- Il modulo Core racchiude la logica di business fondamentale, tra cui il motore di ricerca ibrido (retriever.py), il filtro di protezione PII (pii_filter.py), il re-ranker posizionale (reranker.py), il sistema di deduplicazione (deduplication.py) e il gestore dei dati analitici (analytics.py).

- Il modulo Governance gestisce la persistenza degli utenti, l'assegnazione dei ruoli (RBAC) e la scrittura dei log di audit su file protetti con meccanismi di atomic file locking.

3.3 Frontend in React e TypeScript per un'esperienza utente reattiva

Il frontend di Hermes Knowledge è stato realizzato in React 18 con l'ausilio di TypeScript per garantire la sicurezza sui tipi di dato durante la fase di compilazione e ridurre gli errori a runtime. L'interfaccia utente sfrutta Vite come strumento di build superveloce e Tailwind CSS per un design sistema moderno, fluido, reattivo e conforme alle linee guida di usabilità e accessibilità.

L'interfaccia è strutturata per offrire un'esperienza d'uso intuitiva e professionale:

- Una Sidebar di navigazione consente di selezionare la biblioteca documentale attiva, passare dalla modalità Chat all'esploratore dei documenti, e accedere alla sezione di Amministrazione e Analytics.
- L'area di Chat Interattiva permette di inviare quesiti in linguaggio naturale, visualizzare la risposta generata in streaming asincrono e ispezionare le fonti utilizzate. Ogni citazione include il nome del file, il punteggio di rilevanza calcolato dal re-ranker e un pulsante per scaricare direttamente il documento originale.
- Il pannello di Analytics & Knowledge Gaps fornisce grafici e tabelle riassuntive sull'utilizzo del sistema, mostrando le domande che non hanno trovato risposta per consentire ai manager di integrare la documentazione mancante.

3.4 Storage locale, vettori e gestione isolata delle biblioteche

Per garantire la massima flessibilità e l'assenza di dipendenze da database complessi in fase di prima installazione, Hermes Knowledge adotta un sistema di storage flessibile gestito dal modulo LibraryStore. I documenti caricati dagli utenti vengono salvati all'interno di directory strutturate ed isolati per biblioteca. Gli indici dei vettori ed i metadati associati ai documenti (titolo, autore, data di caricamento, numero di chunk, hash del contenuto) vengono memorizzati in archivi strutturati in formato JSON e SQLite con modalità WAL (Write-Ahead Logging), garantendo alta concorrenza e integrità transazionale.

L'architettura è predisposta per consentire la sostituzione trasparente del motore di storage locale con database vettoriali distribuiti (quali Qdrant o PostgreSQL con estensione pgvector) attraverso l'interfaccia astratta del componente di persistenza.

CAPITOLO 4: IL MOTORE DI RETRIEVAL E GENERAZIONE (RAG)

4.1 Ingestion dei documenti, parsing e strategie di chunking dinamico

Il processo di acquisizione della conoscenza all'interno di Hermes Knowledge inizia con la fase di ingestion dei documenti. Il sistema supporta molteplici formati di file comunemente utilizzati in ambito aziendale, tra cui file PDF, documenti Word (.docx), testi in formato Markdown, file di testo semplice (.txt) e pagine web scaricate tramite lo scraper integrato.

Quando un documento viene aggiunto a una biblioteca, il parser estrae il testo grezzo rimuovendo elementi di formattazione non rilevanti e suddivide il testo in blocchi omogenei denominati chunk. La scelta della dimensione del chunk e del margine di sovrapposizione (overlap) è determinante per la

qualità del RAG: chunk troppo piccoli rischiano di perdere il contesto complessivo della frase, mentre chunk troppo grandi rischiano di diluire l'informazione rilevante e superare il limite di token dell'LLM. Ermes adotta una strategia di chunking dinamico basata su paragrafi e punteggiatura con overlap configurabile per preservare la continuità semantica tra blocchi adiacenti.

4.2 La Ricerca Ibrida: combinazione di vettori e keyword matching (BM25)

Uno dei punti di forza dell'architettura di Ermes Knowledge è l'adozione di un motore di ricerca ibrido. La ricerca puramente vettoriale, basata sul calcolo della somiglianza coseno tra gli embedding della domanda e dei chunk, è straordinariamente efficace nel cogliere il significato semantico e le parafrasi, ma può fallire quando l'utente cerca codici di errore specifici, sigle tecniche, numeri di protocollo o nomi propri che non possiedono una rappresentazione semantica distinta nello spazio vettoriale.

Per superare questa limitazione, Ermes combina in parallelo due tecniche di ricerca:

- Ricerca Vettoriale Semantica: genera l'embedding della query tramite modelli dedicati (es. `nomic-embed-text` o `bge-small-en`) e calcola la distanza angolare con i vettori dei documenti.
- Ricerca Sintattica BM25 / Keyword: esegue l'analisi algebrica della frequenza dei termini (TF-IDF / BM25) per individuare le corrispondenze esatte dei termini chiave.

I risultati delle due ricerche vengono fusi insieme tramite l'algoritmo di Reciprocal Rank Fusion (RRF), assegnando a ciascun documento un punteggio combinato calcolato come la somma dei reciproci dei rispettivi ranghi nelle graduatorie vettoriale e lessicale, garantendo che sia i concetti correlati che i codici esatti emergano ai primi posti della graduatoria di recupero.

4.3 Il Re-ranker posizionale e l'algoritmo di deduplicazione Jaccard

Dopo la fase di fusione iniziale dei candidati, Ermes Knowledge applica due stadi di raffinamento avanzato realizzati nel modulo `core/reranker.py` e `core/deduplication.py`:

Il Re-ranker Posizionale ri-valuta il punteggio di ciascun candidato analizzando la vicinanza spaziale delle parole della domanda all'interno del frammento estratto, la presenza di bi-grammi consecutivi e la corrispondenza con il titolo del documento sorgente. Questo passaggio permette di distinguere un documento che contiene le parole della query disperse in punti lontani del testo da un documento che le contiene esattamente nella stessa frase o sezione.

L'Algoritmo di Deduplicazione Jaccard analizza i testi dei candidati estratti calcolando l'indice di somiglianza Jaccard (rapporto tra cardinalità dell'intersezione e dell'unione degli insiemi di parole) sugli shingle di parole. Se all'interno della stessa biblioteca esistono più documenti identici o quasi identici (es. differenti revisioni dello stesso file o copie duplicate), il sistema identifica il gruppo di duplicati e presenta all'utente un unico estratto rappresentativo con il punteggio di confidenza più alto, evitando di saturare il prompt dell'LLM con informazioni ridondanti.

4.4 Generazione controllata e vincolo all'evidenza documentale

I frammenti di testo selezionati dal re-ranker e sanitizzati dal filtro PII vengono inseriti all'interno di un prompt di contesto strutturato, insieme alle istruzioni di sistema per il modello linguistico. Le istruzioni impongono al modello di attenersi rigorosamente ed esclusivamente al testo fornito nel contesto per formulare la risposta.

Se il contesto fornito non contiene informazioni sufficienti per rispondere in modo completo al quesito dell'utente, il modello è istruito a dichiarare esplicitamente la mancanza di elementi, indicando quali aspetti specifici non trova nel testo. In questo modo, l'output generato risulta essere una sintesi fedele, tracciabile e verificabile dell'evidenza documentale.

CAPITOLO 5: SICUREZZA, DATA LOSS PREVENTION (DLP) E GOVERNANCE

5.1 Isolamento delle biblioteche e controllo degli accessi (RBAC)

La sicurezza e il controllo degli accessi costituiscono una componente primaria nell'architettura di Hermes Knowledge. In contesti aziendali, non tutti gli utenti devono avere accesso all'intera conoscenza dell'organizzazione: i documenti amministrativi e contabili devono rimanere riservati alla direzione, i manuali tecnici devono essere accessibili al team operativo, e le procedure generali devono essere visibili a tutti i dipendenti.

Hermes implementa un sistema di Role-Based Access Control (RBAC) articolato su tre ruoli principali:

- **Admin:** possiede i diritti completi di configurazione del sistema, gestione degli utenti, creazione ed eliminazione di biblioteche documentali e consultazione dei log di audit complessivi.
- **Editor:** può caricare, aggiornare o rimuovere documenti all'interno delle biblioteche per le quali possiede l'autorizzazione.
- **Viewer:** può effettuare ricerche ed interrogare l'assistente virtuale solo ed esclusivamente sulle biblioteche a cui è stato espressamente abilitato.

L'isolamento tra biblioteche è applicato a livello di query backend prima ancora di effettuare la ricerca vettoriale o sintattica: una richiesta inviata da un utente verso una biblioteca per la quale non possiede le autorizzazioni viene bloccata all'endpoint API restituendo un errore HTTP 404 (Not Found), garantendo che l'utente non possa nemmeno dedurre l'esistenza o il nome della biblioteca riservata.

5.2 Il filtro PII Guard: mascheramento in tempo reale di dati sensibili

Un'altra funzionalità distintiva di livello enterprise sviluppata nel progetto è il modulo PII Guard (`core/pii_filter.py`). Il Data Loss Prevention (DLP) è un requisito essenziale per la conformità al GDPR e per prevenire la fuga accidentale di dati personali o credenziali di accesso all'interno delle conversazioni AI.

Il filtro PII Guard intercetta il testo sia in fase di caricamento dei documenti sia durante la formulazione della domanda da parte dell'utente, eseguendo una scansione ad alte prestazioni basata su espressioni regolari avanzate e algoritmi di validazione matematica.

5.3 Validazione algoritmica di Carte di Credito (Luhn), IBAN, Codici Fiscali e Token

A differenza dei filtri PII tradizionali che utilizzano semplici espressioni regolari generando un alto numero di falsi positivi (oscurando ad esempio numeri di telefono o codici di prodotto legittimi), Hermes Knowledge integra verifiche algoritmiche rigorose:

Per le Carte di Credito, il sistema identifica le sequenze numeriche di 13-19 cifre e applica l'algoritmo di Luhn (modulus 10 checksum). L'algoritmo raddoppia il valore di ogni seconda cifra partendo da destra verso sinistra; se il raddoppio supera 9, viene sottratto 9. Si sommano quindi tutte le cifre: solo se il totale è esattamente divisibile per 10 la carta è riconosciuta come autentica e sostituita con il tag [CARTA_CREDITO]. Numeri casuali o sequenze arbitrarie vengono preservati intatti.

Per le Coordinate Bancarie IBAN, il sistema analizza la struttura del codice ed esegue la verifica algebrica Modulo 97 (ISO 13616), convertendo le lettere in numeri e calcolando la divisione modulare per 97. Solo se il resto finale è esattamente 1, l'IBAN è strutturalmente valido e viene sostituito con il tag [IBAN].

Per i Codici Fiscali Italiani, il sistema verifica sia la corrispondenza con il pattern alfanumerico ufficiale di 16 caratteri sia la coerenza del carattere di controllo finale (CIN), calcolato mediante tabelle di conversione per posizioni pari e dispari.

Il filtro rileva e maschera inoltre Token JWT, Chiavi API (es. prefissi sk-live-, Bearer tokens), Indirizzi Email e numeri di telefono, garantendo che nessuna informazione riservata venga mai memorizzata nei log o inviata ai modelli linguistici.

5.3-bis Modello delle minacce e verifica per tentativo di violazione

Le sezioni precedenti descrivono i controlli implementati. Questa descrive il metodo con cui sono stati verificati, che è la parte più significativa del lavoro sulla sicurezza: ogni controllo è stato messo alla prova tentando deliberatamente di aggirarlo, e ogni aggiramento riuscito è stato prima riprodotto con un test automatico e solo dopo corretto.

Il documento [docs/THREAT_MODEL.md](#) elenca le minacce considerate, la difesa attiva per ciascuna e - dichiarati esplicitamente - i punti in cui non esiste ancora una difesa. I difetti reali emersi da questo metodo, tutti coperti oggi da test che fallirebbero se la correzione venisse rimossa, comprendono:

- **Verifica della firma dei token SSO assente (T2):** i token OIDC venivano decodificati senza validarne la firma con le chiavi pubbliche del provider, quindi un token costruito a mano concedeva privilegi amministrativi. La verifica avviene ora tramite JWKS, con soli algoritmi asimmetrici ammessi - perché accettare HS256 consente l'attacco di confusione d'algoritmo, in cui la chiave pubblica del provider viene usata come segreto simmetrico.
- **Tre percorsi di immissione documenti su tre con controlli diversi (T1b, T1c):** oltre al caricamento dal browser esistono un connettore per cartelle di rete e un gateway per automazioni esterne. Entrambi ignoravano le protezioni del primo; dal gateway un utente in sola lettura poteva immettere documenti, che diventano l'evidenza che il sistema cita agli altri utenti come autorevole.
- **Il cruscotto analitico attraversava il confine fra biblioteche (T1d):** la panoramica, accessibile a qualunque utente autenticato, elencava gli identificativi di ogni biblioteca privata con il rispettivo volume di domande.
- **Autenticazione facoltativa su un webhook (T2b):** il controllo del segreto veniva eseguito solo se l'intestazione era presente, quindi una richiesta priva di credenziali passava.
- **Traversal di percorso nel ripristino dei backup (T7b):** l'estrazione dell'archivio scriveva ai percorsi dichiarati nell'archivio stesso, senza verificare che restassero all'interno della cartella dell'applicazione.

- **Password nota nella prima installazione:** seguendo la procedura di avvio documentata, un'installazione appena clonata accettava le credenziali `admin` / `CHANGE\_ME`, e nessuno dei tre controlli previsti lo segnalava.

L'elemento metodologico che emerge da questo elenco è un difetto ricorrente, non una serie di casi isolati: un componente corretto e verificato in isolamento, i cui test passano, che non è collegato a ciò che dovrebbe governarlo. Il limitatore di frequenza delle richieste, per esempio, era completo e coperto da dieci test verdi mentre non era applicato ad alcuna rotta: i test verificavano la classe, non il servizio. Per questa ragione le prove di regressione introdotte interrogano gli endpoint reali anziché le funzioni, e due controlli automatici aggiuntivi verificano che ogni impostazione dichiarata sia effettivamente letta da qualcuno e che ogni segreto che protegge una rotta pubblica sia documentato.

5.4 Audit Logging e tracciamento delle operazioni conforme al GDPR

Tutte le operazioni rilevanti svolte all'interno del sistema (autenticazione degli utenti, creazione di biblioteche, caricamento o eliminazione di documenti, esecuzione di query e modifiche ai ruoli) vengono registrate in un Audit Log ad accodamento, in cui ogni voce è firmata con una chiave HMAC generata per singola installazione.

La formulazione corretta è "manomissione rilevabile", non "registro immutabile": un file su disco resta modificabile da chi ha accesso alla macchina, ma qualunque alterazione invalida la firma della voce e viene segnalata dalla verifica. La distinzione non è accademica - durante lo sviluppo la chiave di firma si è rivelata prima pubblica (un valore predefinito presente nel codice, che rendeva falsificabile qualunque voce) e poi soggetta a sovrascrittura in caso di errore transitorio di lettura, cioè a distruzione silenziosa dell'unico elemento che rende il registro verificabile. Entrambi i difetti sono documentati in [docs/THREAT_MODEL.md](#) alla voce T6, insieme alla prova di regressione che li copre.

Ogni voce di log contiene il timestamp ISO 8601 dell'evento, l'identificativo dell'utente o della chiave API, l'indirizzo IP di provenienza, la tipologia di azione eseguita e l'esito dell'operazione. La scrittura dei log avviene su file protetti con meccanismi di concorrenza atomica (FileLock), impedendo che accessi simultanei possano corrompere il registro. Questa tracciabilità completa soddisfa le richieste dei revisori di sicurezza ed è indispensabile per la conformità agli standard ISO 27001 e SOC 2.

CAPITOLO 6: QUALITÀ DEL CODICE, VALUTAZIONE E TESTING

6.1 L'importanza della verifica quantitativa: il Golden Set di valutazione

Un principio fondamentale seguito durante lo sviluppo di Ermes Knowledge è che la qualità di un sistema RAG non può essere valutata in modo soggettivo o basandosi su impressioni estemporanee, ma deve essere misurata quantitativamente attraverso metriche ripetibili ed oggettive.

A questo scopo, è stato creato un Golden Set di valutazione (`evaluation/library_gold_set.json`), contenente un insieme curato di domande di test classificate in tre categorie concettuali:

- **Query Dirette:** domande formulate utilizzando le medesime parole chiave presenti nel documento sorgente.

- Query Parafrasate: domande che esprimono lo stesso concetto ma utilizzano vocaboli e strutture sintattiche totalmente differenti rispetto alla fonte.
- Query di Astensione: domande verosimili ma aventi ad oggetto argomenti deliberatamente assenti dal corpus documentale.

La misurazione sistematica delle prestazioni del motore di retrieval su queste categorie consente di verificare la capacità del sistema di recuperare i documenti giusti e di astenersi correttamente quando l'informazione non è presente.

6.1.1 Risultati della misurazione

I valori seguenti sono prodotti da comandi contenuti nel repository e riproducibili da chiunque lo cloni ([python evaluation/run_library_eval.py](#)). Sono riportati integralmente, compresi i risultati sfavorevoli, perché sono la ragione per cui alcune funzionalità sono disattivate per impostazione predefinita.

Recall@3 su 27 domande, tre configurazioni a confronto:

| Categoria | Configurazione predefinita | Con ricerca semantica | Con verifica dell'evidenza |

|---|---|---|---|

| Query dirette | **1.000** | 1.000 | 1.000 |

| Query parafrasate | 0.500 | **0.875** | 0.625 |

| Astensione corretta | **1.000** | 0.000 | **1.000** |

La lettura di questa tabella è più istruttiva del singolo numero migliore. La ricerca semantica migliora sensibilmente la comprensione delle parafrasi (da 0.500 a 0.875), ma azzerla la capacità di astenersi: la somiglianza vettoriale trova sempre *qualcosa* di vagamente affine, e quel qualcosa viene citato come evidenza. Le due proprietà sono in conflitto, e la scelta di quale privilegiare è una decisione di prodotto, non un dettaglio di implementazione: per un sistema che dichiara di rispondere solo con evidenza, l'astensione ha precedenza, e la ricerca semantica resta disattivata per impostazione predefinita.

Velocità su un archivio di dimensione da ufficio ([python evaluation/archive_scale.py](#)):

| Passaggi indicizzati | Indicizzazione | Ricerca tipica | Ricerca nel caso peggiore |

|---|---|---|---|

| 10.000 | 11,5 s | 1,4 ms | 281 ms |

| 50.000 | 68,8 s | **3,2 ms** | 3,3 s |

6.1.2 Il limite dell'astensione, misurato

Il risultato meno favorevole dell'intera valutazione riguarda proprio la promessa centrale del sistema. Sul corpus dimostrativo l'astensione è perfetta (1.000); aggiungendo un centinaio di passaggi di prosa non correlata scende a **0.333**, cioè due domande di astensione su tre ricevono una citazione sicura verso un testo irrilevante.

La causa è stata isolata nel codice: un frammento viene ammesso come evidenza se condivide **un solo termine** con la domanda, e all'inizio ogni termine pesava allo stesso modo. Una domanda di astensione risultava così agganciata a un paragrafo tecnico del tutto estraneo sulla base delle sole parole *sempre*, *senza* e *mai*. L'introduzione di una pesatura per rarità dei termini (IDF) ha migliorato l'ordinamento dei risultati ma **non** ha risolto l'astensione: è stata misurata, non ha prodotto l'effetto atteso, e il risultato negativo è documentato anziché rimosso.

La contromisura efficace è un secondo controllo di natura non statistica: un modello linguistico locale valuta, per ogni frammento candidato, se contenga davvero la risposta alla domanda. Con `ERMES_EVIDENCE_VERIFIER=1` l'astensione torna a 1.000 a tutte le dimensioni di corpus misurate e, sul corpus più grande, migliora anche il recall complessivo. Non è attivo per impostazione predefinita perché richiede un modello raggiungibile: un'installazione che fa affidamento sull'astensione deve abilitarlo.

Un ultimo risultato negativo, riportato per completezza: il re-ranker neurale, previsto come componente di qualità, è stato misurato e **disattivato**, perché peggiorava ogni configurazione provata. La configurazione precedentemente attiva per impostazione predefinita era la peggiore fra quelle confrontate.

6.2 Suite di test automatizzati (Pytest) e copertura delle regressioni

Per garantire la stabilità e la robustezza del codice ad ogni modifica, il progetto integra una suite di test automatizzati realizzata con Pytest, che conta **509 test** unitari e di integrazione, eseguiti a ogni push.

I test coprono in modo esaustivo tutti i moduli critici del sistema:

- La validazione degli algoritmi del filtro PII Guard (verificando che carte di credito valide vengano oscurate e quelle non valide ignorate).
- L'isolamento e la sicurezza degli endpoint API e delle rotte RBAC.
- La correttezza del re-ranker e dell'algoritmo di deduplicazione Jaccard.
- Il funzionamento della sincronizzazione automatica delle cartelle e della persistenza del database.
- Le protezioni di ciascuna delle tre vie di immissione documenti e delle rotte esposte a integrazioni esterne (server MCP, webhook di automazione, bot di chat), verificate interrogando gli endpoint reali.
- Il ripristino dei backup, compresi gli archivi malformati e i tentativi di scrittura fuori dalla cartella dell'applicazione.

L'esecuzione automatica in Continuous Integration su GitHub Actions comprende, oltre ai test, tre controlli bloccanti - analisi statica (`ruff`), verifica dei tipi (`mypy`) e analisi di sicurezza (`bandit`) - più la compilazione del frontend e la costruzione dell'immagine Docker. Il servizio PostgreSQL viene avviato come container di servizio nella pipeline: senza di esso gli otto test di parità fra i due backend si sarebbero saltati da soli, e il doppio backend dichiarato non sarebbe stato eseguito da nessuna parte.

Una nota di metodo, perché riguarda l'affidabilità di quanto sopra: un test che passa non è di per sé una prova. Durante lo sviluppo si sono verificati due casi in cui un test risultava verde per una ragione estranea a ciò che doveva verificare - una cartella presente solo sulla macchina di sviluppo, e un controllo la cui condizione non veniva mai eseguita. In entrambi i casi il difetto è emerso solo

dall'esecuzione su un ambiente pulito. Da questa esperienza deriva la pratica adottata sistematicamente nella parte finale del lavoro: ogni prova di regressione viene prima eseguita contro il codice *non* corretto, per verificare che fallisca, e solo dopo si applica la correzione.

6.3 Rilevazione automatica delle lacune informative (Knowledge Gaps)

Un componente innovativo sviluppato nel modulo `core/analytics.py` è il tracciamento e l'analisi automatica dei Knowledge Gaps. Quando gli utenti effettuano ricerche su argomenti per i quali il sistema restituisce "nessuna evidenza" o quando gli utenti forniscono un feedback negativo a una risposta, il sistema registra l'evento nel modulo di analytics.

L'interfaccia di amministrazione aggrega questi eventi identificando le domande ricorrenti prive di copertura documentale. Questo strumento fornisce un valore strategico inestimabile ai responsabili aziendali e alle risorse umane, evidenziando esattamente quali procedure o documenti mancano nell'archivio aziendale e necessitano di essere redatti o aggiornati.

CAPITOLO 7: ESPERIENZA DI TIROCINIO E IMPATTO AZIENDALE

7.1 Integrazione nel flusso di lavoro aziendale ed esperienza formativa

L'esperienza di tirocinio formativo svolta nell'ambito del percorso ITS ha rappresentato un'occasione fondamentale per applicare le metodologie dell'Ingegneria del Software e dell'Intelligenza Artificiale all'interno di un contesto operativo reale. Durante il periodo di stage, è stato possibile confrontarsi direttamente con le esigenze concrete dell'organizzazione partner, analizzando le dinamiche di gestione della conoscenza e le problematiche legate alla sicurezza dei dati.

L'integrazione nel team di sviluppo aziendale ha permesso di perfezionare l'uso delle metodologie agili (Scrum/Kanban), il versionamento del codice tramite Git, la progettazione di API RESTful e le tecniche di testing automatizzato.

È necessario delimitare con precisione il perimetro del lavoro svolto durante il tirocinio, distinguendolo da quello successivo. Presso l'azienda ospitante, con la supervisione del referente aziendale, è stato realizzato il **prototipo** del sistema: l'idea progettuale, l'impostazione dell'architettura RAG e una prima implementazione funzionante del principio dell'evidenza. Quel prototipo è il risultato del percorso formativo, ed è ciò che il tirocinio ha prodotto.

Tutto quanto descritto nei capitoli 3, 5 e 6 di questa tesina - la riscrittura del backend, l'isolamento delle librerie, il modello delle minacce e le correzioni di sicurezza che ne sono derivate, il doppio backend SQLite/PostgreSQL, le integrazioni esterne e l'intera valutazione quantitativa - è stato sviluppato **per iniziativa personale, dopo la conclusione del periodo di tirocinio**, al di fuori di qualunque commessa o supervisione aziendale. Attribuire quel lavoro al tirocinio sarebbe scorretto verso l'azienda ospitante e, soprattutto, non renderebbe conto di dove è stato speso l'impegno maggiore.

7.2 Impatto operativo dell'adozione di Hermes nell'organizzazione

L'introduzione della piattaforma Hermes Knowledge all'interno dei flussi aziendali produce benefici tangibili su molteplici fronti:

In primo luogo, si registra una drastica riduzione dei tempi di ricerca delle informazioni operative da parte dei dipendenti, che possono interrogare la base di conoscenza aziendale in linguaggio naturale ed ottenere risposte sintetiche corredate dai link diretti ai documenti originali.

In secondo luogo, il mascheramento PII e l'esecuzione locale riducono in modo sostanziale l'esposizione dei dati riservati, perché nessun contenuto documentale lascia il perimetro dell'organizzazione se un amministratore non abilita esplicitamente un modello esterno su una singola biblioteca.

Anche qui la formulazione va tenuta esatta: il sistema fornisce alcune delle basi tecniche richieste dal GDPR - minimizzazione, tracciabilità degli accessi, mascheramento dei dati personali prima dell'invio a un modello - ma **non costituisce di per sé conformità**, che riguarda l'organizzazione e non il software. Due strumenti tecnici che un titolare del trattamento deve avere sono implementati: l'esportazione di tutto ciò che è riferibile a un account (art. 15) e la sua cancellazione (art. 17), con una regola esplicita su cosa si cancella, cosa passa all'organizzazione - le biblioteche di cui la persona era proprietaria contengono documenti aziendali, non dati personali - e cosa si conserva dichiarandolo: le voci del log di audit, che sono un registro di sicurezza firmato voce per voce (art. 17(3)(b)). Ciò che manca ancora, dichiarato nella documentazione, è la conservazione a termine con cancellazione automatica. Infine, il modulo di analisi delle lacune informative fornisce alla direzione uno strumento proattivo per identificare le carenze documentali e migliorare continuamente il patrimonio informativo dell'azienda.

CAPITOLO 8: CONCLUSIONI E SVILUPPI FUTURI

8.1 Risultati raggiunti e considerazioni finali

Il lavoro svolto nel progetto Ermes Knowledge ha mostrato come sia possibile progettare e realizzare un sistema RAG utilizzabile in un contesto aziendale reale, coniugando le potenzialità dell'Intelligenza Artificiale Generativa con i vincoli di riservatezza e tracciabilità che quel contesto impone.

Il risultato più significativo non è però un componente, ma un criterio: ogni affermazione contenuta in questa tesina e nel repository è accompagnata dalla misura che la sostiene, oppure dalla dichiarazione esplicita del suo limite. I risultati sfavorevoli - l'astensione che si degrada al crescere del corpus, il re-ranker disattivato perché peggiorava le prestazioni, una correzione tentata e respinta perché non produceva l'effetto atteso - sono riportati con lo stesso rilievo di quelli positivi. È la differenza fra un sistema che sembra funzionare e un sistema di cui si conoscono i confini.

I risultati raggiunti sono sintetizzabili nei seguenti punti chiave:

- Realizzazione di una piattaforma RAG Local-First completa di frontend moderno in React e backend ad alte prestazioni in FastAPI.
- Sviluppo di un motore di ricerca ibrido (Vettoriale + BM25) potenziato da re-ranking posizionale e deduplicazione automatica dei contenuti.
- Implementazione del modulo PII Guard con validazione algoritmica di Carte di Credito (Luhn), IBAN (Modulo 97) e Codici Fiscali per la protezione dei dati sensibili.
- Isolamento del contesto e controllo degli accessi basato sui ruoli (RBAC), affiancato da un registro di Audit Logging ad accodamento con firma per voce, che rende rilevabile qualunque manomissione.

- Validazione quantitativa delle prestazioni tramite Golden Set, con i risultati riportati integralmente al capitolo 6 - compresi quelli sfavorevoli - e copertura di testing automatizzato con 509 test in Pytest, affiancati in CI da analisi statica, verifica dei tipi e analisi di sicurezza bloccanti.
- Un modello delle minacce documentato, con la verifica condotta per tentativo di violazione: le difese sono state messe alla prova cercando di aggirarle, e ognuno degli aggiramenti riusciti è oggi coperto da una prova di regressione.

8.2 Roadmap di evoluzione Enterprise (Scalabilità Cloud, Qdrant, OpenTelemetry)

La versione attuale è operativa e misurata su archivi di dimensione da ufficio; le direttrici seguenti riguardano l'evoluzione verso la grande scala.

Due voci presenti nella prima stesura di questa roadmap sono state nel frattempo realizzate, e vanno quindi spostate fra i risultati: l'integrazione **Single Sign-On (SSO / OIDC)** con gli Identity Provider aziendali (Microsoft Entra ID, Keycloak, Okta), con verifica della firma dei token tramite le chiavi pubbliche pubblicate dal provider, e il **backend PostgreSQL** alternativo a SQLite, la cui parità di schema è verificata in CI contro un'istanza reale del database.

Restano da affrontare:

- **Indice vettoriale distribuito**: transizione dall'indice locale a un cluster (Qdrant, oppure PostgreSQL con pgvector) per gestire milioni di documenti a latenza sub-secondo.
- **Cache di ricerca condivisa**: sessioni, tentativi di accesso e contatori del limitatore di frequenza vivono già sull'archivio condiviso, quindi più istanze contano insieme; la cache di ricerca resta per processo, il che costa lavoro ripetuto ma non correttezza.
- **Mitigazione del prompt injection**: un documento immesso legittimamente può contenere istruzioni rivolte al modello. Oggi non esiste una contromisura, ed è dichiarato come tale nel modello delle minacce.
- **Conservazione a termine**: cancellazione automatica degli eventi analitici e delle voci di audit oltre un periodo configurato. Esportazione e cancellazione su richiesta sono implementate (capitolo 5).
- Osservabilità Avanzata tramite OpenTelemetry: integrazione di tracciamento distribuito (OTel) per monitorare latenze, metriche computazionali ed utilizzo dei token in tempo reale all'interno di dashboard aziendali Grafana/Datadog.

BIBLIOGRAFIA E SITOGRAFIA

- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).
- Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).
- Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval.

- Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). *Reciprocal rank fusion outperforms condorcet and individual machine learning methods for informative retrieval*. ACM SIGIR.
- FastAPI Documentation & Best Practices (2025). <https://fastapi.tiangolo.com/>
- React 18 & TypeScript Production Patterns (2025). <https://react.dev/>
- ISO/IEC 27001:2022 Information Security, Cybersecurity and Privacy Protection.
- ISO 13616: Financial Services - International Bank Account Number (IBAN).
- Regolamento Generale sulla Protezione dei Dati (GDPR - Regolamento UE 2016/679).
- Ollama Open Source Local LLM Runtime Documentation. <https://ollama.com/>